

Homework: PCA

Principal Component Analysis

In this problem you'll be implementing Dimensionality reduction using Principal Component Analysis technique.

The gist of PCA Algorithm to compute principal components is follows:

- Calculate the covariance matrix X of data points.
- Calculate eigenvectors and corresponding eigenvalues.
- Sort the eigenvectors according to their eigenvalues in decreasing order.
- Choose first k eigenvectors which satisfies target explained variance.
- Transform the original data of shape m observations times n features into m observations times k selected features.

The skeleton for the *PCA* class is below. Scroll down to find more information about your tasks.

```
In [17]: import math
import pickle
import gzip
import numpy as np
import pandas
import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [18]: from sklearn.preprocessing import StandardScaler

class PCA:
    def __init__(self, target_explained_variance=None):
        """
        explained_variance: float, the target level of explained variance
        """
        self.target_explained_variance = target_explained_variance
        self.feature_size = -1

    def standardize(self, X):
        """
        standardize features using standard scaler
        :param X: input data with shape m (# of observations) X n (# of feat
        :return: standardized features (Hint: use sklearn's StandardScaler.)
        """
        # Q1. Standardize X using sklearn's StandardScaler. Read the documen
        # Hint: In the example, they used .fit() and .transform() methods se
        # YOUR CODE HERE
        scaler = StandardScaler()
```

```

X_std = scaler.fit_transform(X)
return X_std

def compute_mean_vector(self, X_std):
    """
    compute mean vector
    :param X_std: transformed data
    :return mean vector of shape (# of features,)
    """
    # Q2. return mean vector of shape (# of features,). Hint: averaging
    # YOUR CODE HERE
    return np.mean(X_std, axis=0)

def compute_cov(self, X_std, mean_vec):
    """
    Covariance using mean, (don't use any numpy.cov)
    :param X_std:
    :param mean_vec:
    :return n X n matrix:: covariance matrix
    """
    # Q3. Caculate covariance matrix https://en.wikipedia.org/wiki/Covariance\_matrix
    # Hint:  $E[(X-\mu)^T(X-\mu)]$ . You can assume equal probability when calc
    # YOUR CODE HERE
    m = X_std.shape[0]
    X_centered = X_std - mean_vec
    cov_matrix = (X_centered.T @ X_centered) / m
    return cov_matrix

def compute_eigen_vector(self, cov_mat):
    """
    Eigenvector and eigen values using numpy. Uses numpy's eigenvalue fu
    :param cov_mat:
    :return: (eigen_values, eigen_vector)
    """
    # Q4. Return eigenvalues and engenvectors.
    # Hint: Use appropriate function in linalg package. https://numpy.org/doc/stable/reference/generated/numpy.linalg.eigh.html
    # YOUR CODE HERE
    eigen_vals, eigen_vecs = np.linalg.eigh(cov_mat)
    return eigen_vals, eigen_vecs

def compute_explained_variance(self, eigen_vals):
    """
    Q5. Sort eigen values and compute explained variance ratio.
    explained variance informs the amount of information (variance)
    can be attributed to each of the principal components.
    :param eigen_vals:
    :return: explained variance ratio.
    """
    # YOUR CODE HERE
    total = np.sum(eigen_vals)
    var_exp = [(i / total) for i in sorted(eigen_vals, reverse=True)]
    return var_exp

```

```

def cumulative_sum(self, var_exp):
    """
    return cumulative sum of explained variance.
    :param var_exp: explained variance ratio
    :return: cumulative explained variance ratio
    """
    return np.cumsum(var_exp)

def compute_weight_matrix(self, eig_pairs, cum_var_exp):
    """
    compute weight matrix of top principal components conditioned on target explained variance.
    (Hint : use cumulative explained variance ratio and target_explained_variance_ratio to find the number of top components)

    :param eig_pairs: list of tuples containing eigenvalues and eigenvectors sorted by eigenvalues in descending order (the biggest eigenvalue is first)
    :param cum_var_exp: cumulative explained variance by features
    :return: weight matrix (the shape of the weight matrix is n X k)
    """
    matrix_w = np.ones((self.feature_size, 1))
    # Q6. In this function, you will implement weight matrix calculation
    # For each iteration, check the cumulative explained variance ratio
    # then add the eigenvector as column to the matrix_w above.
    # matrix_w will have the dimension of (784,1) initially, but each iteration it increases by 1
    # the cumulative explained variance reaches the target explained variance
    # YOUR CODE HERE
    matrix_w = []
    for i, cum_var in enumerate(cum_var_exp):
        eigvec = eig_pairs[i][1].reshape(-1,1)
        if i == 0:
            matrix_w = eigvec
        else:
            matrix_w = np.hstack((matrix_w, eigvec))
        if cum_var >= self.target_explained_variance:
            break
    return matrix_w

def transform_data(self, X_std, matrix_w):
    """
    transform data to subspace using weight matrix
    :param X_std: standardized data
    :param matrix_w: weight matrix
    :return: data in the subspace
    """
    return X_std.dot(matrix_w)

def fit(self, X):
    """
    entry point to the transform data to k dimensions
    standardize and compute weight matrix to transform data.
    The fit function returns the transformed features. k is the number of principal components
    explained variance ratio meets the target_explained_variance.
    :param X: m X n dimension: train samples
    """

```

```

: return m X k dimension: subspace data.
"""

self.feature_size = X.shape[1]

# Multisteps to accomplish the fit function- 16 pts
# step 1. Standardize X to X_std using an appropriate function you implement
X_std = self.standardize(X)
# YOUR CODE HERE

# step 2. get mean_vec and cov_mat from the appropriate functions from the class
mean_vec = self.compute_mean_vector(X_std)
cov_mat = self.compute_cov(X_std, mean_vec)
# YOUR CODE HERE

# step 3. get eigenvalues and eigenvectors from the implemented function
eig_vals, eig_vecs = self.compute_eigen_vector(cov_mat)
# YOUR CODE HERE

# step 4. Sort both eig_vals and eig_vecs by descending order in eig_vals
# For example, the first 5 elements of the sorted eigenvalues would be
# and reorder the eigenvector list accordingly.
# Make a list of tuple called eig_pairs
eig_pairs = [(eig_vals[i], eig_vecs[:,i]) for i in range(len(eig_vals))]
eig_pairs.sort(key=lambda x: x[0], reverse=True)
# Each eigenvector has a dimension of (784,)
# YOUR CODE HERE

# step 5. get explained variance ratio and cumulated explained variance ratio
# Use the variable names below.
var_exp = self.compute_explained_variance(eig_vals)
cum_var_exp = self.cumulative_sum(var_exp)
# YOUR CODE HERE

# This step calculates the matrix_w
matrix_w = self.compute_weight_matrix(eig_pairs=eig_pairs, cum_var_exp=cum_var_exp)

k = matrix_w.shape[1]

# save for reporting
self.var_exp = var_exp
self.cum_var_exp = cum_var_exp
self.k = matrix_w.shape[1]

print(f"Reduced dimension k for target explained variance of {self.target_var}")
return self.transform_data(X_std=X_std, matrix_w=matrix_w)

```

To Do: Complete helper functions above.

complete `fit()` to using all helper functions to find reduced dimension data.

Run PCA on *fashion mnist dataset* to reduce the dimension of the data.

fashion mnist data consists of samples with *784 dimensions*.

Report the reduced dimension k for target explained variance of **0.99**

```
In [19]: X_train = pickle.load(open('./data/fashionmnist/train_images.pkl','rb'))
y_train = pickle.load(open('./data/fashionmnist/train_image_labels.pkl','rb'))

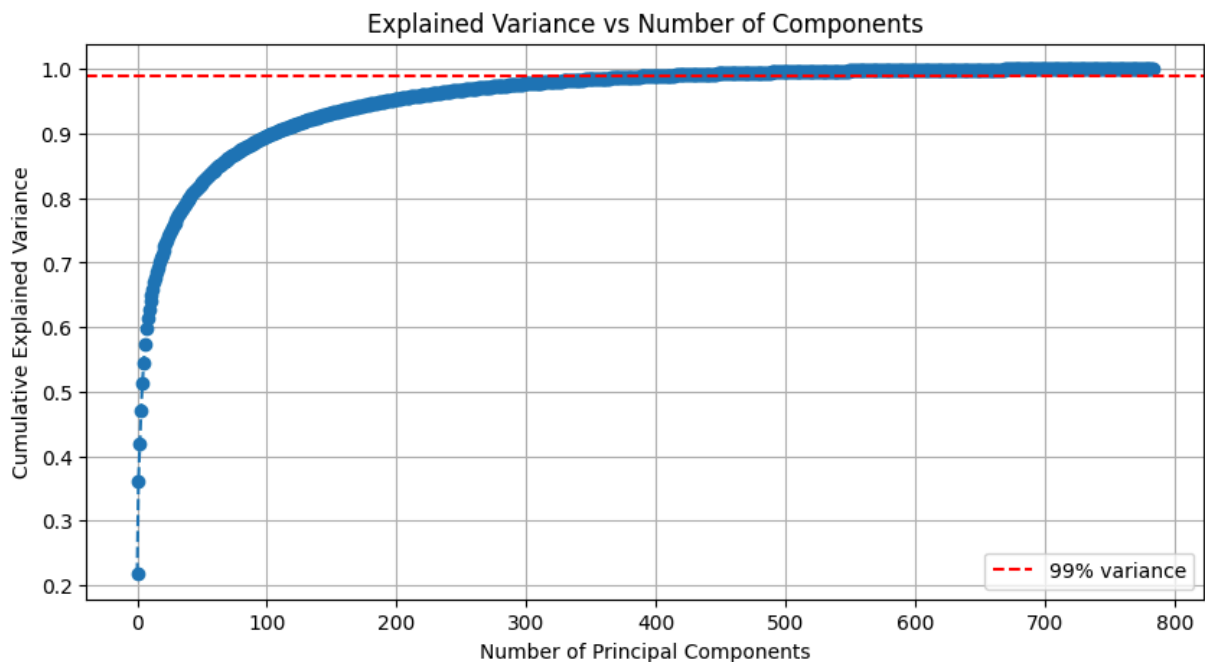
X_train = X_train[:1500]
y_train = y_train[:1500]
```

```
In [20]: pca_handler = PCA(target_explained_variance=0.99)
X_train_updated = pca_handler.fit(X_train)
```

Reduced dimension k for target explained variance of 0.99: 410

```
In [21]: # plot cumulative explained variance

plt.figure(figsize=(10,5))
plt.plot(np.cumsum(pca_handler.var_exp), marker='o', linestyle='--')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance vs Number of Components')
plt.axhline(y=0.99, color='r', linestyle='--', label='99% variance')
plt.grid(True)
plt.legend()
plt.show()
```



```
In [ ]:
```